



第12章 Matplotlib数据可视化

赵晓航

xiaohangzhao@mail.shufe.edu.cn

上海财经大学·信息管理与工程学院



目录

1. Matplotlib概述
2. Matplotlib的绘图基础
3. 基本绘图
4. Pandas可视化



1. Matplotlib概述

- Matplotlib 是 Python 中功能强大且广泛使用的二维绘图库之一，适用于数据科学、工程、金融等领域的多种数据可视化任务。它提供了一整套灵活的工具，可以生成多种类型的静态、动态和交互式图形。
- 在现代数据分析中，数据可视化不仅能够直观呈现数据，还能揭示隐藏的模式和趋势。Matplotlib 通过丰富的图表类型和灵活的定制功能，能够高效完成从基础绘图到复杂分析的多样化任务。无论是基础的**折线图**、**柱状图**，还是更复杂的**散点图**、**热图**，Matplotlib 都能够轻松实现。
- **Matplotlib 的安装与导入**
 - 要使用Matplotlib，首先需要在Python环境中进行安装。通过以下命令完成安装：
 - **pip install matplotlib**
 - 安装完成后，可以在Python程序或交互式环境中导入Matplotlib库的 pyplot 模块：
 - **import matplotlib.pyplot as plt**
 - pyplot 是 Matplotlib 中最常用的模块，提供了一组类似 MATLAB 的简单函数接口，用于快速创建和管理图形。它简化了绘图任务，能够轻松生成各种图表并进行显示。



2. Matplotlib的绘图基础

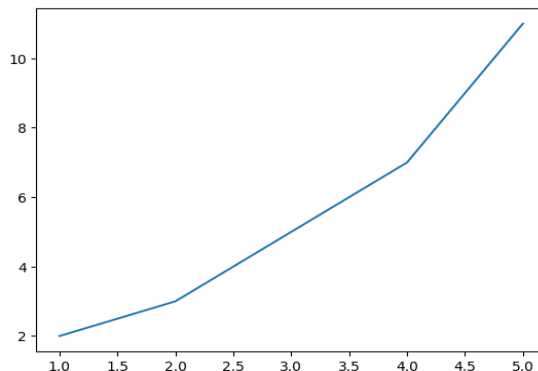
- **Figure和Axes的概念**

- **(1) Figure**: Figure 是整个绘图的容器，类似一张画布，所有图形元素都在 Figure 中绘制。创建 Figure 对象时可以指定画布的大小和分辨率。
 - `import matplotlib.pyplot as plt`
 - `fig = plt.figure(figsize=(8, 6))` # 创建一个 8x6 英寸的 Figure
 - 一个 Figure 对象可以包含多个 Axes 对象，每个 Axes 对象对应一个独立的子图。
- **(2) Axes**: Axes 是 Figure 中的子区域，实际的绘图内容都绘制在 Axes 对象上。Axes 包含两个坐标轴（x 轴和 y 轴），以及标题、标签等附加信息。一个 Figure 可以包含多个 Axes，每个 Axes 可对应不同的子图。
 - `ax = fig.add_subplot(111)`
 - 上述代码中，`add_subplot(111)` 表示在一个 1 行 1 列的网格中添加第 1 个子图。这种方式适合创建单一子图。
 - 若需创建多个子图，可以使用 `add_subplot` 或 `plt.subplots()` 方法来高效管理多个 Axes。
 - `fig, axes = plt.subplots(2, 2)` # 创建一个包含 2x2 网格子图的 Figure



3. 基本绘图

- **折线图** (Line Plot)：是数据可视化中最常用的图表之一，用于展示数据随时间或某一变量的变化趋势。Matplotlib 提供了简单灵活的折线图绘制功能。
- (1) 绘制简单折线图
- 可以使用 `plt.plot()` 函数快速绘制折线图。以下是一个基本示例：
 - `import matplotlib.pyplot as plt`
 - `x = [1, 2, 3, 4, 5]`
 - `y = [2, 3, 5, 7, 11]`
 - `plt.plot(x, y) # 绘制折线图`
 - `plt.show() # 显示图形`
- 在该例中，首先导入了 `matplotlib.pyplot` 模块，然后 `plt.plot(x, y)` 绘制了一条通过数据点 (1,2)、(2,3)、(3,5) 等连接而成的折线。 `plt.show()` 用于显示绘制的图形。





• (2) 自定义折线样式、颜色与轴标签

- 为了使图形更加清晰直观，可以通过参数自定义折线样式、颜色，并添加标题和轴标签。

- `import matplotlib.pyplot as plt`

- `x = [1, 2, 3, 4, 5]`

- `y = [2, 3, 5, 7, 11]`

- `plt.plot(x, y, color='red', linestyle='--', marker='o') # 自定义样式`

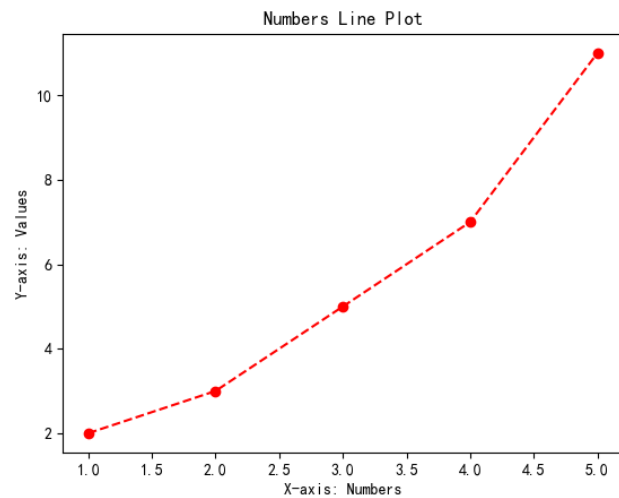
- `plt.title('Numbers Line Plot') # 设置标题`

- `plt.xlabel('X-axis: Numbers') # 设置x轴标签`

- `plt.ylabel('Y-axis: Values') # 设置y轴标签`

- `plt.show()`

- 在该例中，`plt.plot()`方法中的参数`color='red'`指定了折线的颜色，`linestyle='--'`指定了虚线样式，`marker='o'`则在每个数据点上添加了圆形标记。Matplotlib 支持多种颜色（如 'blue', 'green'）和样式（如 '-' 表示实线，':' 表示点线），并可通过 `marker` 参数选择不同的标记形状（如 's' 表示方块，'^' 表示三角形等）。`plt.title()`设置了图形的标题，`plt.xlabel()`和`plt.ylabel()`分别设置了x轴和y轴的标签。这些信息有助于读者更好地理解图形的含义。





- **散点图** (Scatter Plot) : 用于展示两组数据之间的关系, 尤其适合显示数据的分布和相关性。

Matplotlib 提供的 `plt.scatter()` 函数可以快速生成散点图, 并支持多种自定义选项。

- 以下是使用Matplotlib绘制简单散点图并添加注释:

- `import matplotlib.pyplot as plt`

- `x = [1, 2, 3, 4, 5]`

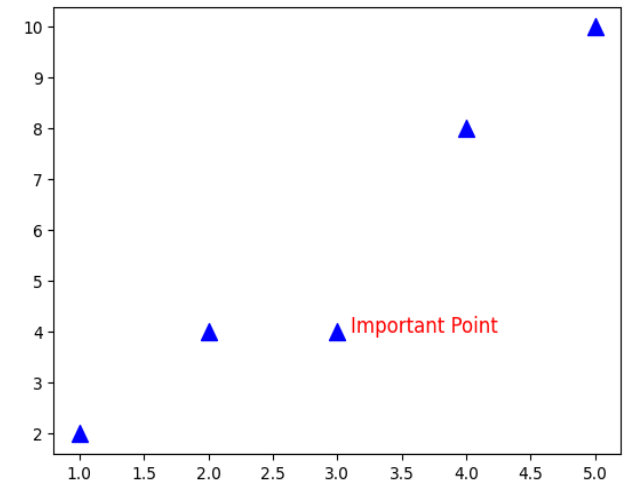
- `y = [2, 4, 4, 8, 10]`

- `plt.scatter(x, y, color='blue', s=100, marker='^') # 绘制散点图`

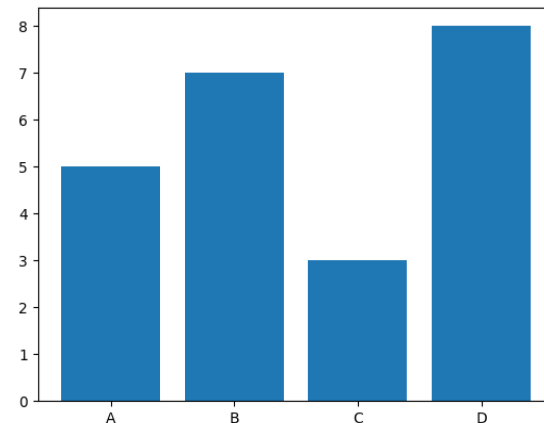
- `plt.text(3.1, 4, 'Important Point', fontsize=12, color='red')`

- `plt.show()`

- 在该例中, `plt.scatter(x, y)`绘制了一个散点图, 每个点表示x和y之间的对应关系, 其中`color='blue'`设置了点的颜色, `s=100`设置了点的大小, `marker='^'`将点的形状改为三角形。`plt.text(3.1, 4, 'Important Point')`在点(3.1, 4) 附近添加了注释"Important Point"。

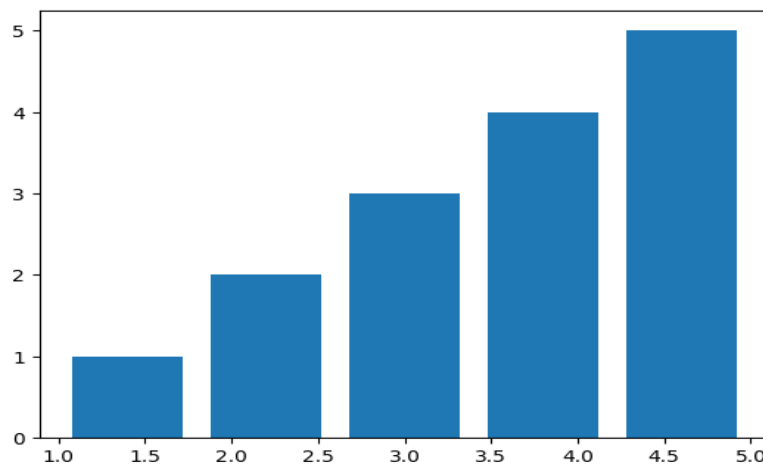


- **条形图**(Bar Plot): 用于比较不同类别的数据值, 是展示分类数据的理想选择。Matplotlib 提供的 `plt.bar()` 和 `plt.barh()` 函数可以分别绘制垂直和水平条形图。此外, 还支持绘制多组条形图, 用于比较不同类别之间的多组数据。
 - (1) 绘制垂直条形图
 - 以下示例展示了如何使用 Matplotlib 绘制一个简单的垂直条形图:
 - `import matplotlib.pyplot as plt`
 - `categories = ['A', 'B', 'C', 'D']`
 - `values = [5, 7, 3, 8]`
 - `plt.bar(categories, values) # 绘制垂直条形图`
 - `plt.show()`
 - 在该例中, `plt.bar(categories, values)` 根据类别 `categories` 和对应值 `values` 绘制垂直条形图, 条形的高度表示数据值, 横轴表示类别。



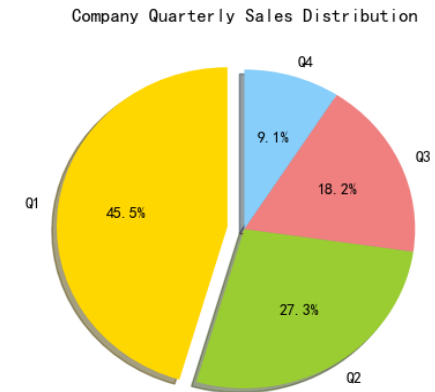


- **直方图**(Histogram): 用于展示数据的分布情况, 特别是展示数据的频率分布。通过 Matplotlib 提供的 `plt.hist()` 函数, 可以生成直方图并进行自定义设置。
 - `import matplotlib.pyplot as plt`
 - `data = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5, 5]`
 - `plt.hist(data, bins=5, rwidth=0.8)` # 绘制直方图
 - `plt.show()`
 - 在该例中, `plt.hist(data, bins=5, rwidth=0.8)`使用5个柱来表示数据的频率分布, `rwidth=0.8`指定了柱的宽度为80%。



- **饼图** (Pie Chart)：用于展示数据各部分在整体中所占的比例，通常用于构成分析，适合反映比例分布。Matplotlib 提供的 `plt.pie()` 函数可以生成饼图，并支持多种自定义选项。。

- `import matplotlib.pyplot as plt`
- `sales = [25000, 15000, 10000, 5000]` # 定义季度销售数据
- `quarters = ['Q1', 'Q2', 'Q3', 'Q4']` # 季度标签
- # 设置颜色和高亮显示特定部分
- `colors = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue']`
- `explode = (0.1, 0, 0, 0)` # 高亮显示Q1
- `plt.pie(sales, explode=explode, labels=quarters, colors=colors, autopct='%1.1f%%', shadow=True, startangle=90)` # 绘制饼图
- `plt.title("Company Quarterly Sales Distribution")` # 公司季度销售分布
- `plt.show()`
- 在该例中，`plt.pie()`函数的参数`explode`用于突出显示某个部分（第一季度），`labels`为每个部分的名称，`autopct='%1.1f%%'`指定了每个部分的百分比显示格式，`startangle=90`指定起始角度，可调整饼图旋转角度。



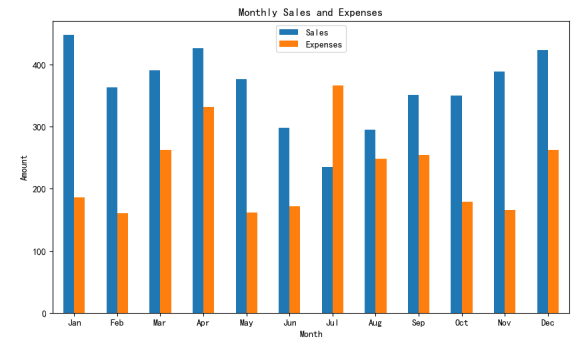


4. Pandas的可视化

- Pandas不仅用于数据处理，还提供了基本的可视化功能。Pandas的数据结构，如DataFrame和Series，可以直接调用绘图方法，利用Matplotlib的底层支持，实现基本的可视化需求。这些绘图方法提供了简洁的接口，能够快速生成折线图、柱状图、直方图等常见图形，方便数据分析的快速展示与探索。

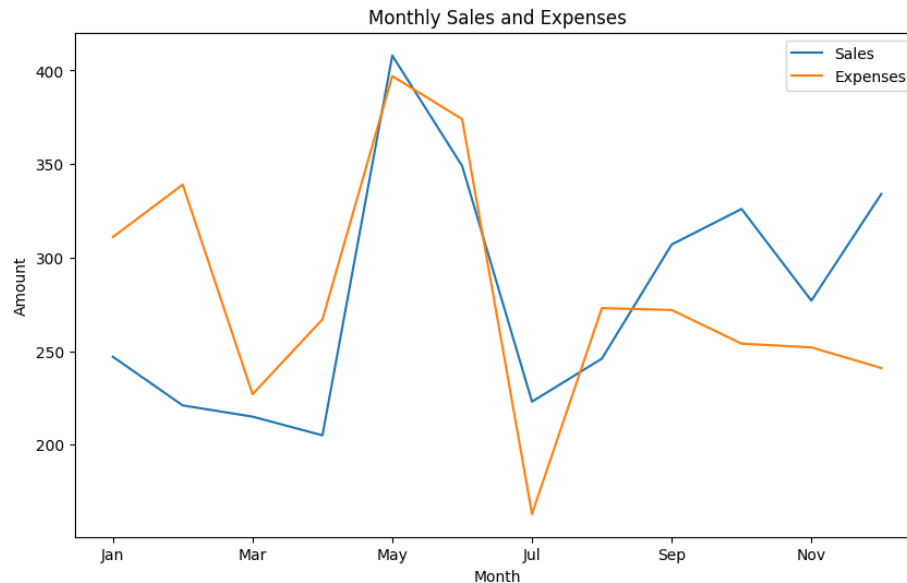
- **示例：**使用 Pandas 的 `plot()` 方法生成柱状图，并对销售额和支出数据进行可视化：

- `import pandas as pd`
- `import numpy as np`
- `import matplotlib.pyplot as plt`
- `data = {`
- `'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'],`
- `'Sales': np.random.randint(200, 500, 12),     # 销售额`
- `'Expenses': np.random.randint(150, 400, 12)   # 支出`
- `}`
- `df = pd.DataFrame(data)`
- `df.set_index('Month', inplace=True) # 设置索引`
- `ax = df.plot(kind='bar', figsize=(10, 6), title="Monthly Sales and Expenses", rot=0) # 使用 Pandas 的 plot 方法绘制柱状图`
- `ax.set_xlabel("Month") # 设置 x 轴标签`
- `ax.set_ylabel("Amount") # 设置 y 轴标签`
- `plt.show()`
- 在上述代码中，首先创建包含月份、销售额和支出数据的 DataFrame 对象，并将月份设为索引，使其成为图形的 x 轴。然后使用 `df.plot(kind='bar')` 方法直接调用柱状图，通过 `kind` 参数指定图表类型为 `bar`，`figsize` 参数设置图表尺寸，`title` 添加图表标题，`rot` 参数调整 x 轴标签的旋转角度。最后通过 `ax.set_xlabel()` 和 `ax.set_ylabel()` 分别设置 x 轴和 y 轴的标签。



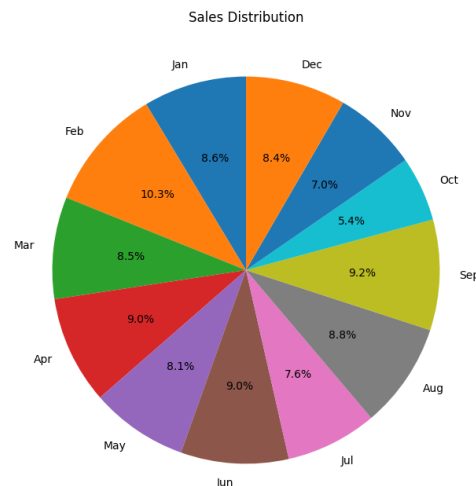
- **示例：**使用 Pandas 的 `plot()` 方法绘制折线图

- # 绘制折线图，只修改上例中的一行代码
- `ax = df.plot(kind='line', figsize=(10, 6), title="Monthly Sales and Expenses", marker='o')` # `kind='line'` 指定折线图
- `kind='line'`：指定图表类型为折线图。`marker='o'`：为每个数据点添加圆形标记。





- **示例：**使用 Pandas 的 `plot()` 方法绘制饼图，通过 `kind='pie'` 参数绘制饼图。注意：饼图通常用来显示单列数据的占比。
 - `# 绘制扇形图`
 - `df['Sales'].plot(kind='pie', figsize=(8, 8), autopct='%1.1f%%', title="Sales Distribution", startangle=90)`
 - `plt.ylabel("")` # 隐藏 y 轴标签
 - `plt.show()`
 - `kind='pie'`：指定图表类型为饼图。`autopct='%1.1f%%'`：在图中显示百分比，格式为小数点后一位。`startangle=90`：调整饼图的起始角度，使第一个部分从 90 度位置开始。





The End !